

obsidian-mcp-router — référence rapide

v0.60.0 · serveur MCP Obsidian multi-vault + plugin Claude Code qui embarque **et lance** le serveur (48 slash commands · 43 outils MCP · 44 skills)

github.com/tboome33/obsidian-mcp-router

Le router en 30 secondes

Un seul process MCP qui connaît **tous tes vaults Obsidian** (locaux + distants). Chaque outil prend un paramètre `vault` (ou utilise le défaut). Plus besoin d'enregistrer un serveur MCP par vault.

- **Depuis la v0.56.1** : le plugin Claude Code **embarque et lance le serveur** — une installation = tout, une mise à jour = tout. Le serveur est déclaré dans le `.mcp.json` racine du plugin via `${CLAUDE_PLUGIN_ROOT}` ; `node_modules` auto-réparés au premier lancement. L'entrée manuelle `~/.claude.json` reste le parcours dev (Claude Desktop a son propre fichier, `claude_desktop_config.json`)
- **Vaults locaux + distants** traités à l'identique — URL + clé API dans la config, terminé
- **Cross-vault** : `vault: "*"` fan-out automatique sur tous les vaults
- **Mode lock** : restreint la session à un seul vault (sécurité, focus) · **Auto-enrichissement** : Claude propose des saves wiki à 3 moments naturels, 4 modes (`ClaudeAsk` / `Hybrid` / `FullAuto` / `off`)
- **Boîte à outils de conversion** (v0.11+) : PDF / DOCX / XLSX / PPTX / image (OCR) / audio / YouTube / page web / dépôt git → markdown — backend **MarkItDown opt-in** depuis v0.56.0 (`npm run install-markitdown` ; les outils restent listés et renvoient un hint d'installation si le backend manque) ; **Docling** second opt-in (`npm run install-docling`) pour le PDF haute fidélité + `pdf_to_images` (le modèle voit une page de PDF) ; **filtre de pertinence BM25** (v0.47) qui retire les blocs hors-sujet avant synthèse
- **Knowledge graph** : graphe JSON typé + parcours de lecture + voisins + plus court chemin de liens ; export en `llms.txt` ou **bundle OKF** (Open Knowledge Format v0.1 de Google, avec l'un des premiers validateurs de l'écosystème)
- **Click-to-open** : liens cliquables dans le chat via la route `GET /open/<path>` du bridge ; `open_in_obsidian` fait pareil côté serveur (sans navigateur — idéal Claude Desktop)
- **Wizard de création de vault** (v0.35+) : `plan_vault` → ajustements → `provision_vault` , défauts d'abord (happy path = 1 interaction)
- **Mode multi-tenant** (v0.9+, opt-in) : scoper une instance par variables d'env — whitelist de vaults, lecture seule, audit d'écriture par utilisateur — pour les déploiements MCPHub/proxy
- **10 hooks** — 2 actifs d'office pour tout installateur du plugin (`hot-cache-load` + `decisions-recall` , via `hooks/hooks.json`) ; les 8 autres opt-in, auto-câblés en fin de bootstrap `setup-vault` (`--no-hooks` pour s'en passer) ou via `node scripts/setup-vault.mjs --install-hooks` : auto-journal de session, garde du hot-cache, autocommit wiki, linter de liens, détecteur de dérive doc, check de mise à jour...

Comment les briques s'emboîtent — la chaîne de dépendances

```
Obsidian ← Local REST API (plugin communautaire) ← BRIDGE (mcp-router-bridge)
  ↑ HTTP par vault (port + apiKey du plugin Local REST API)
SERVEUR MCP (obsidian-mcp-router) – process Node sur le PC
  ↑ MCP sur stdio, spawné par Claude Code
PLUGIN CLAUDE CODE (obsidian-router) – commands + skills + agents + hooks
```

Bridge (v0.7.0) — tourne *dans* Obsidian ; il greffe ses routes (`/search/smart` , `/templates/execute` , `/open/*` , `PUT /vault-cas/*` depuis 0.7.0 — écritures `ifMatch` atomiques) sur le serveur HTTP de Local REST API. **Optionnel** : requis pour `search_smart` , `execute_template` /Templater, les liens click-to-open et les écritures conditionnelles atomiques (sans lui, `ifMatch` retombe sur un repli vérifié non atomique) — le CRUD de base marche toujours. Se met à jour via BRAT depuis les releases GitHub.

Serveur MCP — process Node ≥ 20.18.1 sur le PC, spawné par Claude Code (via le plugin depuis v0.56.1, ou via une entrée manuelle `~/.claude.json` — setups dev). Requier le plugin **Local REST API** dans **chaque** vault (obligatoire : tout le CRUD passe par lui) ; son registre vit dans `~/.claude/obsidian-mcp-router/config.json` .

Plugin Claude Code — ses commands/skills orchestrent les outils MCP du serveur ; deux hooks lisent le vault directement sur disque (`hot-cache-load` , `decisions-recall`). Depuis la v0.56.1 il **embarque et lance le serveur** : une installation = tout, une mise à jour = tout.

Setup d'un vault

1. **Installer le plugin `obsidian-router` dans Claude Code** = le serveur est livré et lancé automatiquement (v0.56+). Le clone + `npm link` (`/obsidian-router:meta-setup`) reste le parcours développeur uniquement.
2. Avoir Node.js ≥ 20.18.1. Outils de conversion : **MarkItDown est opt-in** (`npm run install-markitdown`) · Docling : `npm run install-docling`) — Python 3.10+ requis seulement pour ces extras ; sans backend, les outils renvoient un hint d'installation actionnable.
3. **Chemin le plus simple** : `/obsidian-router:meta-attach-vault` — wizard interactif qui attache un vault à un workspace (cas courant), bootstrappe un vault autonome, ou enregistre un vault distant. Provisionne les plugins + scaffold le wiki + lie le `.env` + picker de conventions.
4. Alternative CLI :

```
npm run setup-vault -- "C:\VAULTS\MonVault"
```

⇒ installe/configure les plugins **Local REST API** + bridge, écrit le `.env` , alloue un port unique, ajoute au registre, câble les hooks routeur — 10 au total, dont 2 déjà actifs via le plugin (`--no-hooks` pour s'en passer, `--install-hooks` pour le faire plus tard).

5. Le **bridge** (`obsidian-mcp-router-bridge`) est requis pour `search_smart` (avec **Smart Connections**), pour `execute_template` (avec **Templater**) et pour les liens click-to-open (`/open/*`). Sans bridge : CRUD de base OK, ces trois capacités non.
6. Vérifier : `/obsidian-router:meta-status` ou dire simplement « *diagnostique le router* »

Configuration — variables d'environnement (`.env` du workspace)

<code>VAULT_PATH</code>	Chemin du vault courant. Auto-détection : s'il correspond à un vault enregistré, ce vault devient le défaut. Écrit par <code>setup-vault.mjs</code> .
<code>OBSIDIAN_ROUTER_DEFAULT_VAULT</code>	Override explicite du vault par défaut pour ce process. Gagne sur <code>VAULT_PATH</code> et sur <code>config.defaultVault</code> . C'est aussi le lien workspace↔vault utilisé par les hooks wiki-query-first.
<code>OBSIDIAN_ROUTER_LOCKED</code>	Verrouille le router sur un seul vault pour la session. Tout appel vers un autre vault échoue. Posé via <code>/obsidian-router:lock <vault> --persist</code> .
<code>OBSIDIAN_ROUTER_AUTO_ENRICH</code>	Mode d'auto-enrichissement wiki : <code>ClaudeAsk</code> (défaut) <code>Hybrid</code> <code>FullAuto</code> <code>off</code> . Posé via <code>/obsidian-router:auto-mode <Mode> --persist</code> .
<code>OBSIDIAN_ROUTER_ALLOWED_VAULTS</code>	Multi-tenant (v0.9+) : whitelist des vaults visibles par cette instance (séparés par des virgules). Les autres sont skippés.
<code>OBSIDIAN_ROUTER_READONLY</code>	Multi-tenant : valeur truthy (<code>true</code> / <code>1</code> / <code>yes</code> / <code>on</code>) désactive les 11 outils d'écriture (<code>write_file</code> , <code>append_to_file</code> , <code>patch_file</code> , <code>set_frontmatter</code> , <code>merge_frontmatter</code> , <code>move_file</code> , <code>delete_file</code> , <code>execute_template</code> , <code>download_page_assets</code> , <code>build_wiki_graph</code> , <code>provision_vault</code>) — filtrés de ListTools ET refusés à l'appel.
<code>OBSIDIAN_ROUTER_USER_ID</code>	Multi-tenant : piste d'audit — chaque écriture réussie ajoute une ligne attribuée au <code>wiki-meta/journal.md</code> du vault touché. Masque aussi les outils local-only <code>plan_vault</code> + <code>provision_vault</code> (déploiements gated/multi-tenant).
<code>VAULT_<NOM></code>	Un vault entier défini dans une variable d'env (JSON : <code>name</code> , <code>baseUrl</code> , <code>apiKey</code> ...) — éditable depuis le dashboard sur les déploiements MCPHub.
<code>OBSIDIAN_ROUTER_NO_WATCH</code>	Désactive le hot-reload du fichier de config (utile en debug).
<code>OBSIDIAN_ROUTER_NO_<HOOK></code>	Opt-outs par hook : <code>NO_HOT_CACHE_LOAD</code> · <code>NO_DECISIONS_RECALL</code> · <code>NO_WIKI_AUTOCOMMIT</code> · <code>NO_SESSION_JOURNAL</code> — chacune désactive le hook correspondant ; acceptent <code>1</code> / <code>true</code> / <code>yes</code> / <code>on</code> .

Fichiers importants — où vivent les choses

<code>~/.claude/obsidian-mcp-router/config.json</code>	Config principale du router (vaults, registre de ports, <code>defaultVault</code> , <code>disabledVaults</code> , <code>remoteVaults</code>). Hot-reload actif.
--	--

<code>~/.claude.json</code>	Plus obligatoire depuis v0.56 : le plugin déclare le serveur (clé <code>router</code>) dans son <code>.mcp.json</code> racine via <code>\${CLAUDE_PLUGIN_ROOT}</code> — PAS en inline dans <code>plugin.json</code> (Claude Code 2.1.220 ignore cette forme). L'entrée manuelle <code>mcpServers.obsidian-router</code> = parcours dev ; garder les deux fait tourner deux process aux outils dupliqués — certains postes dev le font exprès ; <code>claude --plugin-dir <checkout></code> remplace le plugin installé pour une session.
<code>~/.claude/settings.json</code>	Les 8 hooks opt-in (sur 10) vivent ici — les 2 plugin-actifs (<code>hot-cache-load</code> + <code>decisions-recall</code>) viennent de <code>hooks/hooks.json</code> DANS le plugin (<code>hooks.example.json</code> garde un placeholder <code><router-repo></code> car <code>\${CLAUDE_PLUGIN_ROOT}</code> ne s'expande que dans les fichiers du plugin). Le plugin lui-même s'active par workspace via <code><workspace>/ .claude/settings.json</code> — 48 commandes + 44 skills ≈ 10k tokens de contexte, à ne charger que là où tu utilises Obsidian.
<code><workspace>/ .env</code>	Variables d'environnement du workspace courant (ci-dessus). Auto-chargées au boot du router.
<code><vault>/CLAUDE.md</code>	Consigne wiki du vault — chargée en contexte par Claude Code. Contient la consigne d'auto-enrichissement (4 modes) + les conventions installées.
<code><vault>/wiki-meta/{catalog,journal,hot,overview}.md</code>	Scaffolds du LLM-wiki façon Karpathy (catalogue, journal append-only, cache chaud, vue d'ensemble) — sous <code>wiki-meta/</code> depuis v0.12 (les pages utilisateur vivent sous <code>wiki/</code>). Renommés <code>index→catalog</code> / <code>log→journal</code> en v0.58 car OKF réserve ces deux basenames ; les anciens noms restent lus. Scaffoldés par <code>/obsidian-router:wiki</code> .
<code><vault>/wiki-meta/Sessions/</code>	Journaux détaillés par session (écrits automatiquement par le hook <code>session-auto-journal</code> ; <code>log.md</code> reste un index mince).
<code>obsidian-mcp-router/docs/</code>	<code>auto-enrichment.md</code> (4 modes + 4 canaux de placement), <code>features/</code> (13 fiches en prose), <code>architecture.md</code> , ce PDF (FR + EN).

48 slash commands /obsidian-router:<nom> · auto-déclenchement en langage

naturel (FR + EN)

 16 wrappers MCP — un par outil de base du vault

Catégories : discover · read · write · manage · template · convert

COMMANDE	EFFET	PHRASE DÉCLENCHÉUSE (FR/EN)
discover-list-vaults	Liste les vaults (online/latence/ état du lock)	« liste mes vaults » / "list my vaults"
discover-list-files	Liste les fichiers d'un dossier de vault	« liste les fichiers de X » / "list files in Sessions"
read-get	Lit un fichier (markdown + frontmatter)	« montre-moi X » / "show me X"
read-search	Recherche texte simple (substring)	« trouve <texte> » / "grep for <text>"
read-search-smart	Recherche sémantique (Smart Connections)	« trouve mes notes sur X » / "find notes about X"
read-frontmatter	Lit le frontmatter (objet ou une clé)	« quel est le statut de X » / "metadata of X"
write-create-or-replace	PUT — crée ou remplace un fichier	« crée une note X » / "save this as X.md"
write-append	POST — ajoute à la fin d'un fichier	« ajoute à X » / "append to my journal"
write-patch	PATCH chirurgical (heading/block/frontmatter)	« édite la section X dans Y » / "edit X section in Y"
write-frontmatter-set	Pose une clé de frontmatter	« passe le statut de X à closed » / "tag this with X"
write-frontmatter-merge	Pose plusieurs clés en séquence	« sur X mets status=closed outcome=tp1 »
manage-move	Déplace ou renomme un fichier	« renomme X en Y » / "move X to Y"
manage-delete	Supprime un fichier (garde confirm en 2 étapes)	« supprime X » puis « oui confirm=true »

template-execute	Exécute un template Templater	« exécute le template daily » / "run X template"
pdf-to-markdown	PDF local → markdown (MarkItDown, texte brut rapide)	« convertis ce PDF en markdown » / "convert this PDF"
pdf-to-markdown-docling	PDF → markdown haute fidélité (Docling, tableaux/mise en page, opt-in)	« conversion haute fidélité » / "convert with docling"

3 commandes d'état du router (lock + auto-enrichissement)

COMMANDE	EFFET	PHRASE DÉCLENCEUSE (FR/EN)
lock	Restreint la session à un vault (volatile ou <code>--persist</code>)	« verrouille sur X » / "I only want to work on X"
unlock	Lève le lock, restaure le routage multi-vault	« déverrouille les vaults » / "unlock vaults"
auto-mode	Règle le mode d'auto-enrichissement (ClaudeAsk / Hybrid / FullAuto / off)	« passe en mode Hybrid » / "switch to Hybrid mode"

7 helpers conversationnels (setup + diagnostic)

COMMANDE	EFFET	PHRASE DÉCLENCEUSE (FR/EN)
meta-setup	Installation manuelle du router (clone + npm link) — parcours développeur ; depuis v0.56 le plugin installe le serveur	« installe le router » / "setup obsidian-mcp-router"
meta-attach-vault	Wizard — attache un vault à un workspace (courant), autonome, ou distant	« attache un vault à ce workspace » / "connect my remote vault"
meta-status	Health-check de tous les vaults avec pistes de fix	« diagnostique le router » / "are my vaults reachable"
meta-sync-template	Propage plugins/snippets/docs du vault de référence (picker interactif)	« synchronise le template » / "sync the template to all vaults"
sync-from-github	Met à jour un vault ou toute la flotte directement depuis le squelette GitHub (plugins,	« synchronise depuis GitHub » / "sync this vault from github"

thèmes, snippets, docs) — sans
repo de développement local

meta-audit-bridge-readiness	Audite le click-to-open (bridge, REST API, HTTP, probe /open live)	« audite le bridge » / "is click-to-open ready"
conventions	Installe/retire/liste/propage les conventions CLAUDE.md entre vaults	« installe la convention X » / "install source-type on X"



21 commandes de gestion de connaissances (LLM-wiki façon Karpathy)

COMMANDE	EFFET	PHRASE DÉCLENCHEUSE (FR/EN)
wiki	Scaffold le wiki dans un vault (index, log, hot, overview)	« scaffold un wiki » / "set up a wiki"
wiki-ingest	Ingère une source → pages entité/concept + cross-refs	« ingère cette URL » / "absorb this article"
wiki-query	RAG 3 tiers sur le wiki (sans web)	« que dit mon wiki sur X » / "based on my notes ..."
wiki-lint	Health check (orphelins, liens morts, dérive d'index)	« lint le wiki » / "audit my wiki"
wiki-fold	Rollup idempotent du log sous wiki/folds/	« compacte le journal » / "fold the log"
hot-compact	Recompacte un hot.md hors limite vers son contrat de cache (backup vérifié d'abord)	« compacte le hot » / "compact the hot cache"
save	Archive la conversation en note wiki typée (session/décision/ADR/...)	« sauvegarde ça » / "save this"
decision-consolidate	Comprime une page de décision tranchée à l'essentiel et déplace l'historique complet de la délibération vers une note d'archive vérifiée	« consolide cette décision » / "consolidate this decision"
autoresearch	Boucle autonome web→synthèse→archivage	« fais une recherche web sur X » / "research X online"
canvas	Crée/édite des fichiers .canvas Obsidian	« crée un canvas pour X » / "create a canvas for X"
defuddle	Retire le bruit des pages web avant ingestion	« nettoie cette page » / "defuddle <url>"
obsidian-bases	Crée/édite des fichiers .base Obsidian (vues database)	« crée une base pour X » / "create a base for X"
wiki-graph	Construit le knowledge-graph JSON typé (alimente le viewer natif)	« construis le graphe » / "build the wiki graph"

wiki-tour	Parcours de lecture pédagogique ordonné depuis la topologie de liens	« par où je commence » / "give me a tour of this vault"
wiki-neighbors	Voisines d'une page — liens sortants, backlinks, ou les deux	« voisins de X » / "what links to X"
wiki-path	Plus courte chaîne de liens entre deux pages	« quel rapport entre X et Y » / "how is X connected to Y"
wiki-export	Exporte le vault en llms.txt / llms-full.txt ou en bundle OKF	« exporte le wiki » / "export the wiki as llms.txt"
okf-export	Exporte un sous-ensemble du wiki en bundle OKF v0.1 partageable (conformité auto-vérifiée)	« publie en bundle OKF » / "export this folder as an OKF bundle"
okf-projections	Régénère la navigation OKF générée dans wiki/ (index racine + par répertoire, log newest-first) ; auto-rafraîchie après écritures ; -check = rapport de dérive	« régénère les index du wiki » / "refresh the OKF projections"
okf-check	Valide n'importe quel bundle OKF contre les règles de conformité v0.1	« ce bundle est-il conforme ? » / "validate this OKF bundle"
wiki-refresh-digests	Régénère les digests sidecar par page (concepts/claims/keywords)	« rafraîchis les digests » / "refresh the digests"
who-is-speaking	Identifie le membre de la famille dans un vault partagé, lock le routage par membre	« c'est Karine » / "who is speaking"

Les 43 outils MCP — ce que Claude appelle réellement sous le capot

<code>list_vaults · list_files</code>	Découverte. Catalogue des vaults avec état online + latence (à appeler en premier) ; listing de dossier.
<code>get_file · search · search_smart · get_frontmatter</code>	Lectures. Fichier complet (renvoie <code>contentSha256</code> — le jeton <code>ifMatch</code>) ; recherche substring ; recherche sémantique (embeddings Smart Connections, scores cosinus) ; frontmatter typé. <code>vault: "*" = fan-out cross-vault.</code>
<code>write_file · append_to_file · patch_file · delete_file · set_frontmatter · merge_frontmatter · move_file</code>	Écritures & gestion de fichiers. Créer/remplacer ; append ; patch chirurgical par heading/block/frontmatter ; suppression gardée (<code>confirm: true</code>) ; une ou plusieurs clés de frontmatter ; déplacer/renommer. <code>ifMatch (v0.60)</code> : rejouez le <code>contentSha256</code> de <code>get_file</code> et l'écriture est refusée (409) si le fichier a changé depuis votre lecture — atomique avec bridge $\geq$ 0.7.0, repli vérifié sinon.
<code>execute_template</code>	Templater. Rend un template, écrit optionnellement le résultat ; arguments via <code>tp.mcpTools.prompt("clé")</code> .
<code>lock_vault · unlock_vaults · set_auto_enrich_mode</code>	État du router. Isolation mono-vault ; bascule du mode d'auto-enrichissement.
<code>plan_vault · provision_vault</code>	Provisionnement de vault (v0.35+). Plan read-only (défauts + questionnaire + avertissements) → création en un appel (plugins, port, clé API, scaffold wiki, registre). Local uniquement.
<code>pdf/docx/xlsx/pptx/image/audio_to_markdown</code>	Conversion locale (Markdown — opt-in depuis v0.56.0 : <code>npm run install-markitdown</code> ; les outils restent listés et renvoient un hint d'installation si le backend manque). Bureautique/PDF/OCR image/transcription audio → markdown ; chaîner avec <code>write_file</code> .
<code>pdf_to_markdown_docling · pdf_to_images</code>	PDF haute fidélité (Docling opt-in) : mise en page + structure de tableaux. <code>pdf_to_images</code> rend les pages en PNG pour que le modèle les voie (pages/octets plafonnés).
<code>youtube/bing_search/webpage_to_markdown · git_repo_to_markdown</code>	Conversion distante. URL → markdown (garde SSRF) ; transcripts YouTube ; dépôt git → bundle markdown unique (repomix, compression Tree-sitter optionnelle). <code>webpage_to_markdown</code> accepte <code>relevanceQuery</code> (BM25).

<code>filter_relevant_blocks</code>	Filtre de pertinence (v0.47). 2 ^e passe BM25 déterministe sur du markdown déjà acquis — retire les blocs hors-sujet, garde frontmatter/titres, plusieurs garde-fous no-op. Aucun fetch, aucun LLM.
<code>extract_page_metadata</code> · <code>propose_linked_sources</code> · <code>download_page_assets</code>	Support d'ingestion web. Métadonnées de page déterministes (JSON-LD/OpenGraph) ; candidats de liens à suivre scorés ; téléchargement d'images dans le vault.
<code>get_wiki_context_pack</code> · <code>build_wiki_graph</code> · <code>build_wiki_tour</code> · <code>get_page_neighbors</code> · <code>wiki_path</code>	Contexte & graphe. Enveloppe de contexte structurée pour agents externes ; knowledge-graph typé ; parcours de lecture ; voisins par page ; plus court chemin de liens.
<code>build_open_link</code> · <code>open_in_obsidian</code>	Navigation. Liens click-to-open prêts à coller ; ouverture côté serveur (fenêtre Obsidian au premier plan, ancre de titre optionnelle) — sans aller-retour navigateur.

Astuces · Tape `/obsidian-router:` dans Claude Code → autocomplete · Chaque commande accepte un argument `vault` (les wrappers retombent sur le défaut) · `vault: "*" sur search / search_smart = fan-out cross-vault · Les phrases déclencheuses sont indicatives — Claude reconnaît les variantes · Les résultats d'écriture portent un clickToOpenUrl tout fait — un clic ouvre la note dans Obsidian · Préfixes runtime : mêmes outils, trois enregistrements — plugin : mcp__plugin_obsidian-router_router__<tool> · enregistrement manuel : mcp__obsidian-router__<tool> · MCPHub : mcp__<id>__obsidian-router-<vault>-<tool> ; les deux premiers coexistent (outils dupliqués, pas de dédup) si l'entrée manuelle est conservée · Doc complète : README.md , docs/features/ (13 fiches), docs/auto-enrichment.md dans le repo.`